INTERNSHIP PROJECT REPORT

ON

MOVIE DATASET ETL PIPELINE AND SQL ANALYTICS

A report submitted in partial fulfillment of the requirements for the completion of a 45-day internship

Submitted By

[STUDENT NAME]

Enrollment No.: [ENROLLMENT NUMBER]

Course / Semester: [COURSE AND SEMESTER]

Under the Guidance of

[GUIDE / SUPERVISOR NAME]

[COLLEGE / UNIVERSITY NAME]

[DEPARTMENT NAME]

Internship Duration: 45 Days

[MONTH YEAR]

CERTIFICATE

This is to certify that **[STUDENT NAME]**, Enrollment No. **[ENROLLMENT NUMBER]**, student of **[COURSE / DEPARTMENT]** at **[COLLEGE / UNIVERSITY NAME]**, has successfully completed a 45-day internship project entitled **"Movie Dataset ETL Pipeline and SQL Analytics"** under my guidance and supervision.

The work presented in this report is based on the practical activities carried out during the internship. The project involved data extraction, data cleaning, relational table design, Snowflake data loading, and SQL-based analysis of a movie dataset.

Internship Supervisor	Head of Department
Signature: _____	Signature: _____
Name: [SUPERVISOR NAME]	Name: [HOD NAME]
Date: _____	Date: _____

DECLARATION

I hereby declare that the internship report entitled **“Movie Dataset ETL Pipeline and SQL Analytics”** is an original record of the work carried out by me during the 45-day internship period. The project work, analysis, SQL queries, and documentation included in this report were prepared for academic submission.

I further declare that this report has not been submitted previously, in whole or in part, for the award of any other qualification. Wherever external tools, libraries, documentation, or datasets were used, they have been acknowledged appropriately.

Place: [CITY]

Date: [DATE]

Signature of Student: _____

Name: [STUDENT NAME]

ACKNOWLEDGEMENT

I express my sincere gratitude to my internship supervisor, **[SUPERVISOR NAME]**, for providing guidance, feedback, and encouragement throughout the internship. Their support helped me understand the practical application of SQL, data warehousing, and ETL processes.

I am thankful to the faculty members of **[DEPARTMENT NAME]** and **[COLLEGE / UNIVERSITY NAME]** for providing the academic support and resources required to complete this project.

I also acknowledge the open-source community and the documentation of Python, pandas, Airbyte, Snowflake, and SQL, which were useful during the development and testing of the pipeline.

[STUDENT NAME]

ABSTRACT

This report presents the design and implementation of a movie dataset ETL pipeline developed during a 45-day internship focused on SQL and data engineering. The project starts with a raw CSV file containing approximately 4,000 movie records and transforms the data into structured relational tables suitable for analysis.

The pipeline uses Python and pandas for extraction and transformation, Airbyte for source-to-warehouse ingestion, Snowflake as the cloud data warehouse, and SQL for table creation, joins, filtering, grouping, and aggregation. The original denormalized dataset is divided into four related tables: MOVIES, RATINGS, FINANCIALS, and PEOPLE. A generated MOVIE_ID connects the tables and enables meaningful relational queries.

The SQL analysis produces insights such as the highest-rated movies, profitable movies, average rating by genre, yearly financial performance, active directors, and highly voted movies. The project demonstrates the complete flow from raw data to analytical results and provides a foundation for future improvements such as data quality testing, incremental loading, scheduling, and dashboard integration.

TABLE OF CONTENTS

1. Introduction

2. Internship Objectives

3. Background and Technologies Used

4. System Analysis and Requirements

5. Methodology and Pipeline Design

6. Implementation

7. SQL Analysis and Results

8. Testing and Validation

9. Learning Outcomes

10. Limitations and Future Scope

11. Conclusion

References

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The Movie Dataset ETL Pipeline is a data engineering and SQL analytics project. Its primary purpose is to convert a raw movie dataset into clean, structured, and queryable data. Instead of keeping all information in one flat file, the project separates movie details, ratings, financial information, and people involved in the movie into related tables.

The project was developed as an internship assignment over 45 days. The major focus of the internship was SQL, including relational table design, joins, filtering, aggregation, and analytical query writing. Python and pandas were used to prepare the data, while Snowflake was used as the target data warehouse.

1.2 Problem Statement

A raw CSV file is convenient for collection but is not ideal for repeated analysis. It may contain missing values, inconsistent data types, repeated information, and multiple business concepts in the same row. The problem addressed in this project is to build a repeatable process that cleans the raw data, organizes it into related tables, loads it into a warehouse, and supports analytical SQL queries.

1.3 Scope of the Project

- Read a semicolon-separated movie CSV file using the correct encoding.
- Remove records that do not contain a movie name.
- Convert numeric fields such as year, score, votes, budget, and gross.
- Generate a unique movie_id for relational connections.
- Split the source into four subject-oriented tables.
- Create Snowflake tables and load processed data.
- Use SQL joins, filters, GROUP BY, HAVING, ORDER BY, and aggregate functions.

CHAPTER 2: INTERNSHIP OBJECTIVES

2.1 Main Objectives

- Understand the complete ETL lifecycle: extract, transform, and load.
- Develop practical skills in SQL query writing and relational data analysis.
- Learn how a cloud data warehouse such as Snowflake stores analytical data.
- Use pandas to clean and reshape a real-world dataset.
- Understand how Airbyte can automate source-to-destination data movement.
- Design tables that can be joined using a common key.
- Document the workflow so that the pipeline can be reproduced.

2.2 Internship Work Plan

Period	Major Activities
Days 1-5	Project understanding, dataset inspection, requirements, and environment setup.
Days 6-12	CSV extraction, delimiter and encoding handling, and raw data profiling.
Days 13-22	Pandas transformation, data type conversion, missing-value handling, and movie_id generation.
Days 23-30	Relational table design and creation of MOVIES, RATINGS, FINANCIALS, and PEOPLE tables.
Days 31-37	Snowflake loading, Airbyte configuration, and connection verification.
Days 38-43	SQL joins, filters, aggregations, and result validation.
Days 44-45	Documentation, report preparation, review, and final presentation.

CHAPTER 3: BACKGROUND AND TECHNOLOGIES USED

3.1 Python and pandas

Python is used as the programming language for the local ETL workflow. The pandas library reads the CSV file into a DataFrame, removes unusable records, converts columns to numeric types, generates movie identifiers, and creates the four output tables.

3.2 SQL

SQL is the central analytical technology used in the internship. It is used to create tables, join related data, filter records, group data, calculate averages and totals, and order analytical results. The project includes separate SQL files for table creation, joins, aggregations, and filters.

3.3 Snowflake

Snowflake is used as the cloud data warehouse. It stores the processed tables and provides an environment in which SQL queries can be executed. The Python Snowflake connector and write_pandas utility support programmatic loading.

3.4 Airbyte

Airbyte is used for data ingestion from a File/CSV source to Snowflake. The configured source uses a semicolon delimiter and ISO-8859-1 encoding. The connection is documented as a full-refresh, overwrite sync because the dataset is static.

3.5 Supporting Tools

Technology	Purpose
Python	ETL orchestration and scripting
pandas	Data cleaning and transformation
SQL	Schema creation and analytical queries
Snowflake	Cloud data warehouse
Airbyte	Data ingestion and synchronization

Technology	Purpose
python-dotenv	Reading environment variables
Git	Source-code version control

CHAPTER 4: SYSTEM ANALYSIS AND REQUIREMENTS

4.1 Existing System

In the existing form, the dataset is maintained as one denormalized CSV file. Movie details, ratings, financial values, and information about directors and stars occur in the same row. While this format is easy to open, it is less suitable for warehouse-style analysis and makes the data structure difficult to extend.

4.2 Proposed System

The proposed system creates a repeatable ETL pipeline. The raw file is read with the required delimiter and encoding, cleaned with pandas, split into four subject-oriented tables, and loaded to Snowflake. SQL queries then combine the tables when a complete movie-level view is required.

4.3 Functional Requirements

- The system shall read the raw movie CSV file.
- The system shall remove rows without a movie name.
- The system shall convert relevant columns to numeric data types.
- The system shall create a unique identifier for each movie record.
- The system shall generate four processed tables.
- The system shall create corresponding Snowflake tables.
- The system shall support analytical SQL queries.

4.4 Non-Functional Requirements

- The pipeline should be repeatable from the project root.
- Credentials should be supplied through environment variables.
- The output tables should have consistent column names.
- The SQL should be readable and separated by analytical purpose.

CHAPTER 5: METHODOLOGY AND PIPELINE DESIGN

5.1 ETL Architecture

```
Raw movies.csv
|
v
Extract: etl/extract.py
|
v
Transform: etl/transform.py
|
+--> movies.csv
+--> ratings.csv
+--> financials.csv
+--> people.csv
|
v
Load: etl/load.py / Airbyte
|
v
Snowflake tables
|
v
SQL analysis
```

Figure 5.1: Overall Architecture of the Movie ETL Pipeline

5.2 Extraction

The extraction step reads data/raw/movies.csv with pandas. The source file is semicolon-separated and encoded using ISO-8859-1. Handling these settings is important because using the default comma delimiter or UTF-8 encoding can result in incorrect columns or character errors.

```
df = pd.read_csv(
"data/raw/movies.csv",
sep=";",
quotechar='"',
encoding="ISO-8859-1"
)
```

5.3 Transformation

The transformation step removes rows without a name, resets the index, and creates movie_id values beginning from 1. Numeric columns are converted with errors coerced to missing values. The cleaned DataFrame is projected into four tables.

5.4 Loading

The loading step reads the processed CSV files, creates Snowflake tables from the schema SQL, converts column names to uppercase, and uses write_pandas to upload each table. Environment variables are used for credentials and connection settings.

5.5 Data Model

Table	Main Columns	Relationship
MOVIES	MOVIE_ID, NAME, GENRE, YEAR, RUNTIME, COUNTRY, COMPANY	Central movie information
RATINGS	MOVIE_ID, SCORE, VOTES, RATING	Connected through MOVIE_ID
FINANCIALS	MOVIE_ID, BUDGET, GROSS	Connected through MOVIE_ID
PEOPLE	MOVIE_ID, DIRECTOR, WRITER, STAR	Connected through MOVIE_ID

CHAPTER 6: IMPLEMENTATION

6.1 Project Structure

```
movie-etl-pipeline/
|-- data/raw/movies.csv
|-- data/processed/
| |-- movies.csv
| |-- ratings.csv
| |-- financials.csv
| `-- people.csv
|-- etl/
| |-- extract.py
| |-- transform.py
| `-- load.py
|-- sql/
| |-- 01_create_tables.sql
| |-- 02_joins.sql
| |-- 03_aggregations.sql
| `-- 04_filters.sql
|-- airbyte/airbyte_setup.md
|-- main.py
`-- requirements.txt
```

Figure 6.1: Project Folder Structure

6.2 Transformation Implementation

```
df = df.dropna(subset=["name"])
df = df.reset_index(drop=True)
df.insert(0, "movie_id", df.index + 1)

df["budget"] = pd.to_numeric(df["budget"], errors="coerce")
df["gross"] = pd.to_numeric(df["gross"], errors="coerce")
df["score"] = pd.to_numeric(df["score"], errors="coerce")
```

The transformation produces four tables containing the same number of movie-level records. The common movie_id preserves the relationship between the tables and supports SQL joins.

6.3 Snowflake Schema

The Snowflake schema SQL creates the MOVIES, RATINGS, FINANCIALS, and PEOPLE tables. The current schema is intentionally simple for an internship project and can later be strengthened with constraints, data types, and staging tables.

6.4 Pipeline Execution

```
pip install -r requirements.txt
python etl/transform.py
python main.py
```

CHAPTER 7: SQL ANALYSIS AND RESULTS

7.1 Join Queries

The join queries reconstruct a complete movie view by combining MOVIES, RATINGS, FINANCIALS, and PEOPLE using MOVIE_ID. This demonstrates how normalized tables can be used together without storing all attributes repeatedly in one table.

```
SELECT m.NAME, m.GENRE, r.SCORE, r.RATING,
f.BUDGET, f.GROSS, p.DIRECTOR
FROM MOVIES m
JOIN RATINGS r ON m.MOVIE_ID = r.MOVIE_ID
JOIN FINANCIALS f ON m.MOVIE_ID = f.MOVIE_ID
JOIN PEOPLE p ON m.MOVIE_ID = p.MOVIE_ID
ORDER BY r.SCORE DESC
LIMIT 20;
```

7.2 Aggregation Queries

Aggregation queries calculate average rating and movie count by genre, total budget and gross by year, the number of movies associated with each director, and average runtime by decade. These queries use COUNT, AVG, SUM, ROUND, GROUP BY, and HAVING.

```
SELECT m.GENRE,
COUNT(*) AS TOTAL_MOVIES,
ROUND(AVG(r.SCORE), 2) AS AVG_SCORE
FROM MOVIES m
JOIN RATINGS r ON m.MOVIE_ID = r.MOVIE_ID
GROUP BY m.GENRE
ORDER BY AVG_SCORE DESC;
```

7.3 Filter Queries

Filtering queries identify movies released after 2010 with ratings above 7.5, big-budget movies that earned less than their budget, movies from a selected country and genre, and highly voted R-rated movies.

7.4 Key Analytical Questions

Question	SQL Concept Used
Which movies have the highest scores?	JOIN, ORDER BY, LIMIT
Which movies made a profit?	JOIN, arithmetic expression, WHERE
What is the average score per genre?	AVG, COUNT, GROUP BY

Question	SQL Concept Used
Which directors have made the most movies?	COUNT, GROUP BY, HAVING
Which high-budget movies failed commercially?	WHERE and comparison

Table 7.1: SQL Analysis Covered in the Project

CHAPTER 8: TESTING AND VALIDATION

8.1 File and Schema Validation

- The raw CSV contains approximately 4,000 movie records.
- The processed output contains four CSV files.
- Each processed table contains a movie_id column.
- The four tables preserve movie-level row relationships.
- Column names match the Snowflake schema after loading.

8.2 Transformation Validation

- Rows without a movie name are removed.
- Numeric fields are converted using pandas to_numeric.
- movie_id values are generated after resetting the index.
- The data is split without losing the movie-level key.

8.3 SQL Validation

SQL queries were organized into separate files so that schema creation, joins, aggregations, and filters could be tested independently. The query design uses explicit table aliases and join conditions based on MOVIE_ID.

8.4 Current Validation Limitations

The repository does not yet include a formal automated test suite or a complete data-quality framework. In a production implementation, assertions and test cases should verify uniqueness, null rules, valid score ranges, referential integrity, and successful load counts.

CHAPTER 9: LEARNING OUTCOMES

During the internship, the following practical skills were developed:

- Understanding the difference between raw, transformed, and warehouse data.
- Reading CSV files with custom delimiters and character encodings.
- Cleaning missing and incorrectly typed values using pandas.
- Designing related tables from a denormalized dataset.
- Using primary business keys and generated identifiers for joins.
- Writing SQL queries with JOIN, WHERE, GROUP BY, HAVING, ORDER BY, and LIMIT.
- Using aggregate functions such as SUM, AVG, COUNT, and ROUND.
- Understanding cloud warehouse loading through Snowflake.
- Documenting an ETL process for reproducibility.

The most important learning outcome was understanding that SQL analysis becomes more reliable when the data model is designed before queries are written. Separating movie attributes, ratings, financials, and people made the analytical questions clearer and made joins meaningful.

CHAPTER 10: LIMITATIONS AND FUTURE SCOPE

10.1 Current Limitations

- The source dataset is static and the current Airbyte setup uses full refresh.
- The pipeline is manually executed and has no scheduler.
- Formal automated tests and data quality checks are not yet included.
- Snowflake schema constraints and staging-table merge logic can be improved.
- The project currently provides SQL analysis but not a visual dashboard.

10.2 Future Enhancements

- Add incremental synchronization using an update timestamp.
- Use staging tables and MERGE statements to prevent duplicate records.
- Add dbt models, tests, and documentation for warehouse transformations.
- Schedule the pipeline with Airflow, Prefect, or GitHub Actions.
- Add Great Expectations or custom validation checks.
- Connect Snowflake to Power BI or Tableau for interactive dashboards.
- Add monitoring, structured logs, alerts, and row-count reconciliation.

CHAPTER 11: CONCLUSION

The Movie Dataset ETL Pipeline successfully demonstrates an end-to-end data engineering workflow developed during a 45-day SQL-focused internship. A raw movie CSV file was extracted, cleaned, transformed, divided into four related tables, and prepared for loading into Snowflake.

The SQL component of the project provides practical experience with relational data analysis. Joins combine subject-oriented tables, filters isolate relevant records, and aggregation queries produce summarized business insights. The project therefore connects database concepts with a realistic analytical use case.

Although the current implementation is primarily an educational and demonstrative pipeline, it provides a strong foundation for a production-grade solution. Incremental loading, automated tests, scheduling, monitoring, and dashboard integration are the natural next steps.

The internship helped develop a practical understanding of SQL, ETL design, cloud data warehousing, data cleaning, and technical documentation.

REFERENCES

- Python Documentation: https://docs.python.org/3/
- pandas Documentation: https://pandas.pydata.org/docs/
- Snowflake Documentation: https://docs.snowflake.com/
- Airbyte Documentation: https://docs.airbyte.com/
- Project source files: README.md, etl/, sql/, and airbyte/airbyte_setup.md

APPENDIX A: PROJECT FILES

File	Purpose
main.py	Runs extraction, transformation, saving, and loading in sequence.
etl/extract.py	Reads the raw movie CSV.
etl/transform.py	Cleans data and splits it into four tables.
etl/load.py	Creates Snowflake tables and loads processed CSV files.
sql/01_create_tables.sql	Creates the warehouse tables.
sql/02_joins.sql	Contains relational join analysis.
sql/03_aggregations.sql	Contains grouped and summarized analysis.
sql/04_filters.sql	Contains filtered analytical queries.